EDGE INTELLIGENCE LAB TASK – VIDEO PROCESSING

SWETHA K

25MML0008

jupyter 25MML0008_VideoProcessing Last Checkpoint: 29 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

25MML0008 - Swetha K - Video Processing using OpenCV

[1]:
```
pip install opencv-python
```

```
Requirement already satisfied: opencv-python in c:\users\ksksw\anaconda3\envs\ml_lab\lib\site-packages (4.12.0.88)
Requirement already satisfied: numpy<2.3.0,>=2 in c:\users\ksksw\anaconda3\envs\ml_lab\lib\site-packages (from opencv-python) (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

Testing camera module

[2]:
```
import cv2

# Step 1: Initialize camera (0 = default laptop webcam)
cap = cv2.VideoCapture(0)
```

[3]:
```
# Step 2: Check if camera opened successfully
if cap.isOpened():
    print("Camera module detected: True")
else:
    print("Camera module detected: False")
    exit()
```

```
Camera module detected: True
```

Creating video using webcam and saving it using - VideoWriter

[6]:
```
# Step 4: Create VideoWriter object to save video
out = cv2.VideoWriter(
    'edge_vid1.avi',                    # output file
    cv2.VideoWriter_fourcc(*'XVID'),        # codec
    20.0,                                   # frames per second
    (frame_width, frame_height)             # resolution
)

print("Recording... Press 'q' to stop")
# Step 5: Capture video frames
while True:
    ret, frame = cap.read()

    if not ret:
        print("Failed to grab frame")
        break

    # Write frame to file
    out.write(frame)

    # Display the frame
    cv2.imshow('Webcam Video', frame)

    # Press 'q' to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Step 6: Release resources
cap.release()
```

```python
print("Recording... Press 'q' to stop")
# Step 5: Capture video frames
while True:
    ret, frame = cap.read()

    if not ret:
        print("Failed to grab frame")
        break

    # Write frame to file
    out.write(frame)

    # Display the frame
    cv2.imshow('Webcam Video', frame)

    # Press 'q' to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Step 6: Release resources
cap.release()
out.release()
cv2.destroyAllWindows()

print("Video saved as edge_vid1.avi")
```

```
Recording... Press 'q' to stop
Video saved as edge_vid1.avi
```

Getting video properties - Frames, FPS, Height, Width

[14]:

```python
import cv2

cap = cv2.VideoCapture("edge_vid1.avi")

if not cap.isOpened():
    print("Error: Could not open video")
    exit()

# Total number of frames
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Frames per second
fps = cap.get(cv2.CAP_PROP_FPS)

# Frame width
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

# Frame height
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("Video Properties:")
print("Total Frames:", total_frames)
print("FPS:", fps)
print("Frame Width:", frame_width)
print("Frame Height:", frame_height)

cap.release()
```

```python
cap.release()
```

```
Video Properties:
Total Frames: 923
FPS: 20.0
Frame Width: 640
Frame Height: 480
```

Extracting frame from video -> printing sample of original frame -> convert the video to greyscale (frame by frame) -> Print the frame

[7]:
```
import cv2

cap = cv2.VideoCapture("edge_vid1.avi")

if not cap.isOpened():
    print("Error: Could not open video.")
    exit()

ret, frame = cap.read()

if ret:

    # Check frame structure
    print("Original frame shape:", frame.shape)
    print("Sample pixels:")
    print(frame[0:5, 0:5])

    cv2.imshow("Original Frame", frame)

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    print("\nGrayscale frame shape:", gray.shape)
    print("Sample grayscale pixels:")
    print(gray[0:5, 0:5])

    cv2.imshow("Grayscale Frame", gray)
```

```
    cv2.imshow("Grayscale Frame", gray)

    cv2.waitKey(0)
    cv2.destroyAllWindows()

cap.release()
```

```
Original frame shape: (480, 640, 3)
Sample pixels:
[[[172 206 219]
  [173 207 220]
  [173 207 220]
  [174 208 221]
  [174 208 221]]

 [[172 206 219]
  [173 207 220]
  [173 207 220]
  [174 208 221]
  [174 208 221]]

 [[172 206 219]
  [173 207 220]
  [173 207 220]
  [174 208 221]
  [174 208 221]]

 [[172 206 219]
  [173 207 220]
  [173 207 220]
  [174 208 221]
  [174 208 221]]

 [[172 206 219]
  [173 207 220]
```

```
  [174 208 221]
  [174 208 221]]

 [[172 206 219]
  [173 207 220]
  [173 207 220]
  [174 208 221]
  [174 208 221]]

 [[172 206 219]
  [173 207 220]
  [173 207 220]
  [174 208 221]
  [174 208 221]]]

Grayscale frame shape: (480, 640)
Sample grayscale pixels:
[[206 207 207 208 208]
 [206 207 207 208 208]
 [206 207 207 208 208]
 [206 207 207 208 208]
 [206 207 207 208 208]]
```

Saving every 100th frame

[8]:
```
import cv2

cap = cv2.VideoCapture("edge_vid1.avi")

if not cap.isOpened():
    print("Error: Could not open video.")
    exit()

frame_count = 0
saved_count = 0

while True:
    ret, frame = cap.read()

    if not ret:
        break

    frame_count += 1

    # Check if frame is multiple of 100
    if frame_count % 100 == 0:

        filename = f"frame_{frame_count}.jpg"

        # Save the frame
        cv2.imwrite(filename, frame)

        print(f"Saved {filename}")

        # Show the frame
```


```
    # Check if frame is multiple of 100
    if frame_count % 100 == 0:

        filename = f"frame_{frame_count}.jpg"

        # Save the frame
        cv2.imwrite(filename, frame)

        print(f"Saved {filename}")

        # Show the frame
```


```
        cv2.imshow("Extracted Frame", frame)
        cv2.waitKey(500)  # show for 0.5 sec

        saved_count += 1

cap.release()
cv2.destroyAllWindows()

print(f"Total frames saved: {saved_count}")
```

```
Saved frame_100.jpg
Saved frame_200.jpg
Saved frame_300.jpg
Saved frame_400.jpg
Saved frame_500.jpg
Saved frame_600.jpg
Saved frame_700.jpg
Saved frame_800.jpg
Saved frame_900.jpg
Total frames saved: 9
```

Static capturing in the video

[9]:
```
import cv2

# Load one saved frame
img = cv2.imread("frame_100.jpg")

if img is None:
    print("Image not found")
    exit()

# Draw Rectangle
cv2.rectangle(img, (50, 50), (300, 200), (0, 255, 0), 3)

# Draw Circle
cv2.circle(img, (400, 200), 80, (255, 0, 0), 3)

# Draw Line
cv2.line(img, (50, 300), (400, 300), (0, 0, 255), 3)

# Add Text
cv2.putText(img, "Static Capture", (50, 400),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)

# Show the image
cv2.imshow("Shapes on Frame", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Using Canny to detect edges in an image by identifying strong gradients in pixel intensity

[18]:
```
import cv2

# Start webcam
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Camera not detected")
    exit()

print("Press 'q' to quit")

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Optional: reduce noise
    blur = cv2.GaussianBlur(gray, (5,5), 0)

    # Apply Canny edge detection
    edges = cv2.Canny(blur, 100, 200)

    # Show original video
    cv2.imshow("Original Video", frame)
```

```
    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Optional: reduce noise
    blur = cv2.GaussianBlur(gray, (5,5), 0)

    # Apply Canny edge detection
    edges = cv2.Canny(blur, 100, 200)

    # Show original video
    cv2.imshow("Original Video", frame)

    # Show edge detected video
    cv2.imshow("Canny Edge Detection", edges)

    # Press q to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

```
Press 'q' to quit
```

Converting video to negative

[19]:
```python
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    negative = 255 - frame

    cv2.imshow("Negative Video", negative)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Motion Detection

[13]:
```python
import cv2

cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Camera not detected")
    exit()

# Read first frame
ret, prev_frame = cap.read()

while True:
    ret, curr_frame = cap.read()

    if not ret:
        break

    # Convert to grayscale
    prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
    curr_gray = cv2.cvtColor(curr_frame, cv2.COLOR_BGR2GRAY)

    # Blur to reduce noise
    prev_gray = cv2.GaussianBlur(prev_gray, (5,5), 0)
    curr_gray = cv2.GaussianBlur(curr_gray, (5,5), 0)

    # Compute difference between frames
    frame_diff = cv2.absdiff(prev_gray, curr_gray)

    # Threshold to highlight motion
    _, thresh = cv2.threshold(frame_diff, 25, 255, cv2.THRESH_BINARY)
```


```python
    # Blur to reduce noise
    prev_gray = cv2.GaussianBlur(prev_gray, (5,5), 0)
    curr_gray = cv2.GaussianBlur(curr_gray, (5,5), 0)

    # Compute difference between frames
    frame_diff = cv2.absdiff(prev_gray, curr_gray)

    # Threshold to highlight motion
    _, thresh = cv2.threshold(frame_diff, 25, 255, cv2.THRESH_BINARY)
```


```python
    # Edge detection only on motion areas
    edges = cv2.Canny(thresh, 50, 150)

    # Show motion edges
    cv2.imshow("Motion Edges", edges)

    # Update previous frame
    prev_frame = curr_frame

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```